

Protocol Fundamentals & Architecture

1. **Explain the relationship between ARINC 818 and Fibre Channel.** Why does ARINC 818 use the FC physical and link layers?
2. **What are the primary differences between ARINC 818-1 and ARINC 818-2?** (Focus on channel bonding, switching, and higher link rates).
3. **What is an Interface Control Document (ICD) in the context of ARINC 818?** Why is it impossible to design a generic "one-size-fits-all" receiver without it?
4. **Describe the four synchronization classes defined in ARINC 818.** Which one is the most challenging for RTL design?
5. **How does ARINC 818 handle uncompressed vs. compressed video?** How does the protocol indicate that a payload is compressed?

RTL Design & Link Layer (FC-1/FC-2)

6. **Explain the 8b/10b encoding scheme used in ARINC 818.** What is "Running Disparity," and how would you implement a disparity checker in RTL?
7. **What are "Ordered Sets"?** List the specific K-codes used for **SOFn** (Start of Frame Normal) and **EOFn** (End of Frame Terminate).
8. **Describe the ADVB frame header.** What is the significance of the Type field (0xE1) and the SEQ_CNT?
9. **How do you handle the 32-bit CRC calculation?** Is it calculated over the entire frame or just the payload?
10. **Explain the concept of "Idles" (Fill words).** Why are they necessary between ADVB frames, and what happens if the transmitter runs out of data?
11. **Design a State Machine (FSM) for an ARINC 818 Transmitter.** What are the key states from Idle to EOF?
12. **How does the receiver achieve word alignment (Comma alignment)?** Which K-code is typically used for this?

Physical Layer & FPGA Integration

13. **ARINC 818-2 supports rates from 1.0625 to 28.05 Gbps.** If you are using a Xilinx GTH or Intel Stratix 10 transceiver, how do you handle the reference clock requirements for these non-standard rates?
14. **What is "DC Balance," and why is it critical for the high-speed serial links used in ARINC 818?**
15. **How do you manage Clock Domain Crossing (CDC) between the high-speed Transceiver clock and the internal Video Processing clock?**
16. **Explain the importance of the Elastic Buffer in the receiver.** How do you handle overflow/underflow if the TX and RX frequencies have a slight ppm offset?
17. **What is Channel Bonding in ARINC 818-2?** How would you synchronize multiple serial

lanes in RTL to form a single high-resolution image?

Video Mapping & Timing

18. **How is a single video line mapped into ADVB frames?** If a line is too large for the 2112-byte payload limit, how do you split it?
19. **What is "Ancillary Data" in ARINC 818?** Where is it typically placed within the transmission sequence?
20. **Describe the "Container Header."** What information does it provide about the video resolution and pixel format?
21. **How do you handle different pixel formats (RGB, YCbCr, Mono) in RTL?** Do you need separate hardware logic for each?
22. **Explain "Line Synchronous" vs. "Frame Synchronous" timing.** How do you reconstruct the \$Hsync\$ and \$Vsync\$ signals at the receiver?
23. **What is the "Blanking Period" in the context of ADVB?** Are any characters transmitted during this time?

Verification & Debugging

24. **How would you verify an ARINC 818 IP core using a UVM environment?** What would your "Scoreboard" check?
25. **If a link reports "Loss of Sync," what are the first three things you would check in the RTL/Hardware?**
26. **How do you test for CRC errors?** Describe a scenario where a CRC error might occur despite the link being physically stable.
27. **What is the impact of "Jitter" on an ARINC 818 link?** How does it affect the CDR (Clock and Data Recovery) in the receiver?
28. **How do you validate "Low Latency" requirements in an RTL simulation?**
29. **What are the common pitfalls when implementing the ADVB Sequence Count (\$SEQ_CNT\$)?**
30. **Explain how you would implement a "Loopback" test for an ARINC 818 interface on an FPGA.**

I. Protocol Fundamentals & Architecture

1. **Relationship with Fibre Channel:** ARINC 818 (ADVB) uses the **FC-0 (Physical)** and **FC-1 (Data Link)** layers of Fibre Channel. It leverages FC's robust high-speed hardware ecosystem while removing the complex networking overhead (FC-2 through FC-4) to ensure the deterministic, low-latency performance required for avionics.
2. **ARINC 818-1 vs. 818-2:** 818-2 introduced **Channel Bonding** (combining multiple lanes for higher bandwidth), support for **Switching/Routing**, and increased line rates (up to 28 Gbps). 818-2 also refined the use of the Container Header for complex video formats.
3. **The ICD (Interface Control Document):** ARINC 818 is a flexible protocol. The ICD defines the specific **Line Rate, Resolution, Pixel Format, and Timing** for a specific

project. Without it, RTL logic wouldn't know how to sample the incoming serial stream or reconstruct the video frames.

4. **Synchronization Classes:**

- **Class 1:** Full asynchronous (TX and RX independent).
- **Class 2:** Pixel synchronous.
- **Class 3:** Line synchronous.
- **Class 4:** Frame synchronous.
- *RTL Challenge:* **Class 1** is hardest because it requires deep elastic buffers and sophisticated clock recovery to prevent frame drops.

5. **Compression:** ARINC 818 handles compressed video by utilizing the **Type** field in the Object 0 (Header). It indicates whether the payload contains raw pixels or a compressed bitstream (like H.264 or JPEG2000).

II. RTL Design & Link Layer (FC-1/FC-2)

6. **8b/10b & Disparity:** 8b/10b ensures DC balance and enough transitions for clock recovery. **Running Disparity (RD)** tracks the balance of 1s and 0s. In RTL, you implement this using a look-up table (LUT) and a flip-flop to store the current RD (+1 or -1) to choose the next 10-bit symbol.
7. **Ordered Sets:** These are 4-byte sequences starting with a K-code (K28.5).
- **SOFn (Start of Frame):** K28.5, D21.5, D21.2, D21.2.
 - **EOFn (End of Frame):** K28.5, D21.5, D21.4, D21.4.
8. **ADVB Header:** The **Type** field identifies the content (Video, Audio, Ancillary). **SEQ_CNT** is a frame counter (0-255) used to detect dropped frames in the receiver logic.
9. **CRC-32:** Calculated over the **Header and the Payload**. It excludes the SOF and EOF. In RTL, this is usually a parallel LFSR (Linear Feedback Shift Register) processing 32 or 64 bits per clock.
10. **Idles (Fill Words):** (K28.5, D21.4, D21.5, D21.5). These maintain link synchronization and DC balance when no data frames are being sent.
11. **Transmitter FSM:** States: **IDLE** -> **SEND_SOF** -> **SEND_HEADER** -> **SEND_PAYLOAD** -> **CALC_CRC** -> **SEND_EOF** -> **RETURN_TO_IDLE**.
12. **Word Alignment:** The receiver looks for the **Comma character** (the unique 7-bit pattern in K28.5). High-speed transceivers (like Xilinx GTs) have built-in "Comma Alignment" blocks to shift the byte boundary automatically.
-

III. Physical Layer & FPGA Integration

13. **Reference Clocks:** Use a fractional-N PLL or a programmable oscillator (like an Si570). RTL must manage the transceiver's **TXOUTCLK** to drive the user logic to ensure the data rate matches the physical link precisely.

14. **DC Balance:** Prevents charge buildup on AC-coupling capacitors in the differential lines, which would otherwise cause signal distortion and bit errors.
 15. **Clock Domain Crossing (CDC):** Use **Asynchronous FIFOs** with Gray-coded pointers to move data from the transceiver's recovered clock domain to the internal AXI-Stream or video clock domain.
 16. **Elastic Buffer:** A FIFO that absorbs the difference in ppm between the remote TX clock and the local RX clock. If the buffer gets too full/empty during the blanking period, you add or remove "Idle" words to re-center it.
 17. **Channel Bonding:** Multiple lanes are deskewed by looking for a "Master" lane's SOF and delaying the "Slave" lanes in RTL buffers until all lanes align.
-

IV. Video Mapping & Timing

18. **Mapping:** A video line is split into multiple ADVB frames if it exceeds 2112 bytes. For example, a 1920-pixel RGB line (5760 bytes) would be split across 3 frames.
 19. **Ancillary Data:** Metadata (timecodes, overlays, sensor data) is usually sent in **Object 2** frames before the actual video payload (Object 0).
 20. **Container Header:** Provides the "map." It defines the number of lines, pixels per line, and the pixel packing format (e.g., 24-bit RGB).
 21. **Pixel Formats:** Handled by a "Pixel Packer/Unpacker" module. It maps the input video bus (e.g., 32-bit AXI) to the 32-bit ARINC 818 word format based on the ICD.
 22. **Hsync/Vsync Reconstruction:** The receiver uses the EOF and SOF markers combined with a line counter. When the EOF of the last frame of a line is received, the RTL generates a local \$Hsync\$ pulse.
 23. **Blanking Period:** No ADVB frames are sent. The link sends continuous **Idles** to keep the CDR (Clock and Data Recovery) locked.
-

V. Verification & Debugging

24. **UVM Verification:** The Scoreboard would compare the original video frames injected at the TX against the reconstructed frames at the RX, checking for bit-per-bit accuracy and correct \$SEQ_CNT\$ ordering.
25. **Loss of Sync Debugging:**
 1. Check the Reference Clock frequency.
 2. Inspect the Transceiver's **RX_LOCK** signal.
 3. Use an ILA (Integrated Logic Analyzer) to see if K28.5 characters are being detected.
26. **CRC Error Testing:** Intentionally "corrupt" a bit in the RTL TX before the CRC is sent. The RX should flag a CRC error and increment an error counter without crashing the FSM.

27. **Jitter:** Excessive jitter causes the CDR to misinterpret bit boundaries, leading to 8b/10b "Invalid Symbol" errors or "Disparity" errors.
28. **Latency Validation:** Use a timestamp at the SOF generation in the TX and compare it to the SOF detection in the RX using a global simulation timer.
29. **SEQ_CNT Pitfalls:** Ensure the counter wraps from 255 back to 0. A common bug is resetting it at 256 or forgetting to increment it for every Object frame.
30. **Loopback Test:** Connect the FPGA's TX pins to its own RX pins (external loopback) or enable the "Near-End PMA Loopback" inside the transceiver (internal loopback). The RTL then sends a known pattern and verifies it is received correctly.

1. Parallel CRC-32 Implementation

In a high-speed ARINC 818 link, you cannot process one bit per clock cycle (serial CRC). You must process the data at the width of your internal bus (usually 32 bits or 64 bits).

The Logic

The CRC-32 for ARINC 818 uses the polynomial:

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

RTL Design Strategy

- **LFSR Matrix:** You create a combinational matrix that calculates the "next state" of the CRC based on the current 32-bit data word and the current 32-bit CRC register.
- **Initialization:** The CRC register is initialized to **32'hFFFFFFFF** at the Start of Frame (SOF).
- **Final Xor:** After the End of Frame (EOF), the result is typically XORed with **32'hFFFFFFFF** before being sent.

Specific RTL Snippet (Concept)

Verilog

```
// 32-bit Parallel CRC-32 Logic
```

```
always @(posedge clk) begin
```

```
    if (reset || sof_detected) begin
```

```
        crc_reg <= 32'hFFFFFFFF;
```

```
    end else if (data_valid) begin
```

```
        // The 'next_crc' function is a matrix of XORs derived from the polynomial
```

```
        crc_reg <= next_crc32_32bit(crc_reg, current_data_word);
```

end
end

2. 8b/10b Encoder Implementation

The 8b/10b encoder converts 8-bit data into 10-bit symbols to ensure DC balance (equal number of 0s and 1s) and sufficient transitions for the Receiver's Clock and Data Recovery (CDR).

The Logic: Data vs. Control

- **Data (D-codes):** Standard video payload bytes.
- **Control (K-codes):** Used for SOF, EOF, and Idles (e.g., K28.5).

Handling Running Disparity (RD)

This is the "memory" of the encoder. If the last symbol had more 1s than 0s (Positive Disparity), the next symbol must have more 0s than 1s (Negative Disparity) to bring the average back to zero.

RTL Design Components

1. **Look-up Tables (LUTs):** * One 5b/6b encoder.
 - One 3b/4b encoder.
2. **Disparity Logic:** A simple flip-flop stores the `current_rd` (0 for negative, 1 for positive).
3. **Illegal Character Check:** Logic to ensure that if a K-code is requested, it is a valid ARINC 818 character (like K28.5 or K28.7).

3. Timing & Throughput Considerations

When designing these in RTL for ARINC 818-2, you must account for **Backpressure** and **Line Rate**:

- **Zero Latency:** In avionics, every microsecond counts. Your RTL pipeline should be shallow. The CRC calculation must happen in a single cycle so the EOF can be appended immediately after the last data word.
- **Gearshifting:** If your FPGA transceiver provides a 40-bit interface (for 32-bit data + 8-bit overhead) at a different frequency than your video clock, you must use an **Asynchronous FIFO** with "Watermark" logic.
 - *Specific Tip:* If the FIFO reaches a "Low Watermark," the RTL should

automatically insert **Idle Words** (K28.5) to prevent an underflow.

Verification Tip for Interview

If asked how you'd test this specifically: "I would use a **Constrained Random** testbench to inject 'illegal' K-codes and forced 'Disparity Errors' to ensure the Receiver logic triggers the appropriate error flags and doesn't hang the state machine."

RTL Design Deep Dive: Questions & Answers

Q1: How do you implement a CRC-32 generator in RTL when the data arrives at 32 bits per clock cycle instead of 1 bit?

Answer: You must use a **Parallel CRC Architecture**. Instead of a serial shift register, you implement a combinational XOR matrix.

- **The Logic:** You derive equations for each of the 32 bits of the "next CRC state" (\$CRC_{new}\$) as a function of the 32 bits of the "current CRC state" (\$CRC_{curr}\$) and the 32 bits of the "input data" (\$Data_{in}\$).
- **Implementation:** In Verilog, this is a large block of assign statements or a function. The matrix is pre-calculated based on the ARINC 818 (Fibre Channel) polynomial.
- **State Management:** At the **SOF** (Start of Frame), the register is preset to **32'hFFFFFFFF**. On every **data_valid** cycle, the register updates. At the **EOF** (End of Frame), the result is inverted (XORed with **32'hFFFFFFFF**) and appended to the stream.

Q2: In the 8b/10b Encoder, how is "Running Disparity" (RD) handled to ensure DC balance?

Answer: RD is handled using a single-bit state machine (flip-flop) that tracks whether the link is currently "positive" or "negative."

- **The Process:** Most 8-bit characters have two possible 10-bit representations: one with more 0s and one with more 1s.
- **RTL Choice:** The encoder looks at the current RD. If RD is negative, it chooses the 10-bit version that is either neutral or positive to balance the line.
- **Update:** After the 10-bit symbol is sent, the RD state is updated based on the "disparity" of that symbol.
- **Special Case:** K28.5 (the Comma character) is specifically designed to have a unique bit pattern that the receiver can never misinterpret, regardless of disparity transitions.

Q3: How does the RTL handle "Rate Matching" if the incoming video pixel clock is slightly faster than the ARINC 818 line rate?

Answer: This is managed via **Elastic Buffers** and **Idle Insertion/Deletion**.

- **The Buffer:** An asynchronous FIFO sits between the Video side and the ARINC 818

side.

- **Idle Management:** During the horizontal or vertical blanking periods, the RTL monitors the FIFO fill level.
- **The Logic:** If the FIFO is too full (TX side), the RTL sends fewer **Idles** and more data frames. If the FIFO is too empty, the RTL inserts extra **Idle words** (K28.5) to keep the transceiver busy while waiting for the next line of video.

Q4: How do you implement "Channel Bonding" in RTL for an ARINC 818-2 link using four lanes?

Answer: Channel bonding requires a **Deskew FIFO** for each lane.

- **Alignment:** Because of physical trace length differences, the **SOF** may arrive at Lane 0 a few nanoseconds before Lane 1.
- **The Mechanism:** Each lane's receiver writes into its own FIFO. The RTL waits until all lanes have detected the **SOF** marker in their respective FIFOs.
- **The Release:** Once the "All-Lanes-Ready" signal is high, the RTL reads from all FIFOs simultaneously, effectively re-aligning the data into a single wide bus for processing.

Q5: What RTL logic is required to reconstruct the Video Syncs (\$Hsync\$, \$Vsync\$, \$Data_Enable\$) from the ARINC 818 stream?

Answer: This requires a **Timing Generation Module** synchronized to the Frame Headers.

- **\$Vsync\$:** Triggered when a frame with **Object 0** and a specific **Start of Frame** indicator for the first line of a new image is detected.
- **\$Hsync\$:** Generated based on the **EOF** (End of Frame) of the last segment of a video line.
- **\$DE\$ (Data Enable):** High when the RTL is pulling valid pixel data out of the payload section of an **Object 0** frame.
- **Counters:** You must implement a **Line Counter** and **Pixel Counter** that are reset by the ARINC 818 **Container Header** information to ensure the output timing matches the ICD.

Q6: How would you design the RTL to handle a "CRC Error" detected at the Receiver?

Answer: The design must prioritize system stability over corrupted data.

- **Detection:** The Receiver calculates the CRC of the incoming payload in real-time. It compares its result with the 4 bytes immediately preceding the **EOF**.
- **Action:** If they don't match, the RTL asserts an **error_flag** and increments an error counter register (accessible via AXI-Lite for software monitoring).
- **Data Integrity:** Usually, the corrupted frame is "dropped" or the **\$DE\$ (Data Enable)** signal is suppressed for that line to prevent the display from showing "glitched" pixels. The FSM then resets to wait for the next **SOF**

Section 1: Protocol Fundamentals & Architecture

1. Explain the relationship between ARINC 818 and Fibre Channel.

- **Answer:** ARINC 818 is a subset of the Fibre Channel (FC) standard. It uses the **FC-0 (Physical)** and **FC-1 (Data Link)** layers. It strips away the complex networking and handshaking of FC to provide a deterministic, point-to-point "video-centric" protocol.

2. What are the primary differences between ARINC 818-1 and ARINC 818-2?

- **Answer:** 818-2 adds support for **Channel Bonding** (combining lanes for 4K/8K video), higher link rates (up to 28 Gbps), and specific provisions for **Switching/Routing** fabrics within avionics systems.

3. What is an Interface Control Document (ICD) and why is it critical?

- **Answer:** The ICD is the "blueprint" for the link. Because ARINC 818 is highly flexible, the ICD defines the exact **line rate, resolution, color depth, and frame rate**. An RTL core cannot be "generic"; it must be configured based on the ICD to know how to pack and unpack pixels.

4. Describe the four synchronization classes.

- **Answer:** * **Class 1:** Asynchronous (No relationship between TX and RX clocks).
 - **Class 2:** Pixel Synchronous.
 - **Class 3:** Line Synchronous.
 - **Class 4:** Frame Synchronous.
 - *Class 1* is the most common and requires the most robust RTL buffer management.

5. How does the protocol handle compressed vs. uncompressed video?

- **Answer:** It uses the **Type** field in the Object 0 header. A value of **0xE1** typically denotes uncompressed video, while other values (defined in the ICD) point to compressed streams like H.264 or JPEG2000.

Section 2: RTL Design & Link Layer (FC-1/FC-2)

6. Explain 8b/10b encoding and Running Disparity (RD).

- **Answer:** 8b/10b ensures DC balance and sufficient transitions for clock recovery. In RTL, you maintain a 1-bit **rd_state**. If the state is positive, you choose a 10b symbol with more 0s (or neutral) to bring the average back to zero.

7. What are "Ordered Sets" and which K-codes are used for SOF and EOF?

- **Answer:** Ordered Sets are 4-byte sequences starting with a K-code (K28.5).
 - **SOFn:** K28.5, D21.5, D21.2, D21.2.
 - **EOFn:** K28.5, D21.5, D21.4, D21.4.

8. Describe the ADVB frame header.

- **Answer:** It contains the **Type** (payload category), **SEQ_CNT** (8-bit frame counter), and **Payload Length**. The **SEQ_CNT** is vital for RTL to detect if a frame was dropped during transmission.

9. How do you implement 32-bit CRC calculation in RTL?

- **Answer:** You implement a **Parallel CRC matrix** (combinational logic) that processes 32 bits (or 64 bits) of data in one clock cycle using a series of XOR gates based on the FC polynomial.

10. What are "Idles" and why are they sent?

- **Answer:** Idles (K28.5, D21.4, D21.5, D21.5) are sent between frames to maintain link synchronization and DC balance when no active video data is being transmitted.

11. Describe an ARINC 818 Transmitter State Machine.

- **Answer:** States: **RESET** → **IDLE** (send idles) → **SOF_GEN** → **HDR_GEN** → **PAYLOAD_PUSH** (looping) → **CRC_GEN** → **EOF_GEN** → **IDLE**.

12. How does the receiver achieve word alignment?

- **Answer:** It looks for the **Comma** (7-bit unique pattern) inside the K28.5 character. The FPGA transceiver "slides" the byte boundary until the comma is aligned correctly.

Section 3: Physical Layer & FPGA Integration

13. How do you handle non-standard ARINC 818-2 link rates in an FPGA?

- **Answer:** By using a programmable reference clock (like an Si570) and configuring the Transceiver's PLL (CPLL/QPLL) to the specific frequency defined in the ICD.

14. What is "DC Balance" and why is it critical?

- **Answer:** It prevents charge buildup on AC-coupling capacitors. Without it, the voltage baseline shifts, causing bit errors at high speeds.

15. How do you manage Clock Domain Crossing (CDC)?

- **Answer:** Use **Asynchronous FIFOs** with Gray-coded pointers to move data from the transceiver's recovered clock domain (\$RXCLK\$) to the video processing domain.

16. What is the role of the Elastic Buffer?

- **Answer:** It absorbs the frequency drift (ppm) between the TX and RX. The RTL adds or removes "Idles" during blanking to prevent buffer overflow/underflow.

17. What is Channel Bonding?

- **Answer:** It is the alignment of multiple serial lanes. RTL uses a **Deskew FIFO** on each lane, releasing data only when the SOF has been detected across *all* bonded lanes.

Section 4: Video Mapping & Timing

18. How is a video line mapped into ADVB frames?

- **Answer:** If a line exceeds 2112 bytes, the RTL splits the line across multiple ADVB frames. Each frame has its own Header and CRC.

19. What is "Ancillary Data"?

- **Answer:** Non-video metadata (like timestamps or status). It is sent in **Object 2** frames, usually positioned before the video frames.

20. What is the Container Header?

- **Answer:** A special frame (Object 1) that describes the **Video Geometry** (resolution, lines per frame, pixels per line).

21. How do you handle different pixel formats in RTL?

- **Answer:** You design a **Pixel Packer/Unpacker** module that maps standard video buses (like AXI-Stream) into the 32-bit words required by the ARINC 818 payload format.

22. Explain "Line Synchronous" vs. "Frame Synchronous" timing.

- **Answer:** Line Synchronous sends frames immediately after a line is ready. Frame Synchronous sends the entire image in a burst. RTL must adjust its FIFO depth accordingly.

23. What happens during the "Blanking Period"?

- **Answer:** The transmitter sends continuous **Idles**. No data frames are sent, allowing the

receiver to re-center its elastic buffers.

Section 5: Verification & Debugging

24. How would you verify this IP using UVM?

- **Answer:** Create a **Scoreboard** to compare transmitted pixels vs. received pixels and a **Monitor** to check that CRC and SEQ_CNT rules are never violated.

25. If you see "Loss of Sync," what are the first 3 checks?

- **Answer:** 1. Check the Ref Clock. 2. Verify Comma Alignment logic. 3. Check for 8b/10b disparity errors in the Transceiver status.

26. How do you test for CRC errors?

- **Answer:** Use an "Error Injection" module in the testbench to flip a single bit in the payload and verify the receiver flags a CRC error.

27. What is the impact of "Jitter"?

- **Answer:** High jitter causes the Clock and Data Recovery (CDR) to lose track of the bit center, leading to high Bit Error Rates (BER).

28. How do you validate "Low Latency"?

- **Answer:** Measure the time from the first pixel input at the TX to the first pixel output at the RX in simulation using a high-precision timer.

29. Common pitfalls with \$SEQ_CNT\$?

- **Answer:** Incorrectly resetting the counter (it should be 0-255) or forgetting to increment it for Ancillary (Object 2) frames.

30. How do you implement a Loopback test?

- **Answer:** Enable **Near-End PMA Loopback** in the FPGA transceiver. Send a known PRBS (Pseudo-Random Bit Sequence) or video pattern and check for zero errors at the RX.